

=====

TECHNICAL ANALYSIS USING R

DEVELOPMENT PHASE

=====

Submitted By: Paul Kinuthia Kuguru

Student ID: 664421185

Course: BDA400 / Data Science Tools and Techniques

Assignment 5: Technical Analysis using R, Development Phase

Date: June 24, 2026

=====

REPOSITORY LINK

=====

GitHub Repository: https://github.com/pkinuthiak-glitch/Technical-Analysis-using-R-Preliminary-Stage_R

=====

Technical Analysis Indicators Implementation in R

1. Introduction

This document provides a comprehensive overview of nine technical analysis indicators implemented in R. All functions are implemented using only base R functions without any external libraries, following specific templates and pseudocode algorithms.

Key Features:

- No external package dependencies
- Base R functions only
- Input validation included
- Error handling implemented
- Consistent naming conventions
- Clear return structures

2. Simple Moving Average (SMA)

File: sma.R

Description

The Simple Moving Average (SMA) calculates the arithmetic mean of a specified number of data points. It provides a smoothed average of a set of data points over a specified period, giving equal weight to each data point in the calculation.

Formula

text

$$\text{SMA} = (\text{Sum of all data points over the period}) / (\text{Number of data points in the period})$$

Implementation Details

The function:

1. Validates that data length is at least the specified period
2. Creates a sliding window of size period
3. Calculates the mean for each window
4. Returns a vector of SMA values

Parameters:

- data: Numeric vector of data points
- period: Integer representing the number of data points to include in the moving average

Return Value

- Type: numeric vector
- Length: $\text{length}(\text{data}) - \text{period} + 1$
- Description: SMA values for each window

Example

r

```
data <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
```

```
result <- sma(data, period = 3)
```

```
# Output: 12.33333 15.66667 17.66667 20.00000 21.66667 23.66667 23.33333
```

Use Cases

- Trend identification
- Smoothing short-term fluctuations
- Support/resistance levels

3. Exponential Moving Average (EMA)

File: ema.R

Description

The Exponential Moving Average assigns greater weight to recent data points, making it more responsive to recent price movements compared to SMA. This sensitivity helps capture short-term trends and potential reversals.

Formula

text

$\text{Multiplier} = 2 / (\text{period} + 1)$

$\text{EMA}(i) = (\text{Price}(i) - \text{EMA}(i-1)) \times \text{Multiplier} + \text{EMA}(i-1)$

- $\text{EMA}(1) = \text{Price}(1)$ (first value)

Implementation Details

The function:

1. Validates input data length
2. Calculates the smoothing multiplier
3. Sets the first EMA value equal to the first data point
4. Applies the smoothing formula for subsequent points
5. Returns the complete EMA vector

Parameters:

- data: Numeric vector of data points
- period: Integer for the EMA period

Return Value

- Type: numeric vector
- Length: same as input data
- Description: EMA values for all data points

Example

r

```
data <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
```

```
result <- ema(data, period = 3)
```

```
# Output: 10.00000 11.00000 13.00000 16.50000 17.25000 19.62500 22.31250 23.15625  
22.07812
```

Use Cases

- Trend following
- Short-term momentum analysis
- Generating trading signals (crossovers with SMA)

4. Moving Average Convergence Divergence (MACD)

File: macd.R

Description

MACD is a trend-following momentum indicator that shows the relationship between two EMAs of a security's price. It identifies trend direction, momentum, and potential reversal points.

Dependencies

- Requires `ema()` function from `ema.R`

Formula

text

MACD Line = $\text{EMA_short} - \text{EMA_long}$

Signal Line = $\text{EMA}(\text{macd_line}, \text{signal_period})$

Histogram = $\text{MACD Line} - \text{Signal Line}$

Implementation Details

The function:

1. Validates input parameters
2. Calculates short-term EMA
3. Calculates long-term EMA
4. Computes MACD line as the difference
5. Calculates signal line as EMA of MACD
6. Computes histogram as the difference

Parameters:

- `data`: Numeric vector of data points
- `short_period`: Integer for short-term EMA
- `long_period`: Integer for long-term EMA
- `signal_period`: Integer for signal line EMA

Return Value

- Type: list
- Components:
 - o `macd_line`: numeric vector

- o signal_line: numeric vector
- o histogram: numeric vector

Example

r

```
data <- c(100, 105, 110, 115, 120, 125, 130)
```

```
result <- macd(data, short_period = 3, long_period = 5, signal_period = 2)
```

Output: List with macd_line, signal_line, and histogram

Use Cases

- Trend identification
- Momentum analysis
- Buy/sell signals (crossover)
- Divergence detection

Interpretation

Signal	Interpretation
MACD crosses above Signal	Bullish (Buy)
MACD crosses below Signal	Bearish (Sell)
Positive Histogram	Bullish momentum
Negative Histogram	Bearish momentum

5. Standard Deviation (stdev)

File: stdev.R

Description

Standard deviation measures the dispersion of data points from the mean. A smaller standard deviation indicates data points are closer to the mean, while a larger value indicates more spread.

Formula

text

$$\text{Variance} = \sum (x - \text{mean})^2 / n$$

$$\text{Standard Deviation} = \sqrt{\text{Variance}}$$

Implementation Details

The function:

1. Calculates the mean of the data
2. Computes differences from the mean
3. Squares the differences
4. Calculates variance (mean of squared differences)
5. Takes the square root to get standard deviation

Parameters:

- data: Numeric vector of data points

Return Value

- Type: numeric
- Description: Population standard deviation

Example

r

```
data <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
```

```
result <- stdev(data)
```

Output: 4.946629

Use Cases

- Volatility measurement

- Risk assessment
- Normalizing data
- Identifying outliers

Note

This calculates the population standard deviation (dividing by n), not sample standard deviation (dividing by n-1).

6. Linear Regression (linreg)

File: linreg.R

Description

Linear regression identifies the best-fitting straight line through data points, helping to determine trend direction, strength, and make predictions.

Formula

text

$$\text{Slope} = \Sigma((x - \bar{x})(y - \bar{y})) / \Sigma(x - \bar{x})^2$$

$$\text{Intercept} = \bar{y} - \text{Slope} \times \bar{x}$$

$$\text{Predicted} = \text{Slope} \times x + \text{Intercept}$$

Implementation Details

The function:

1. Validates input parameters
2. Calculates start and end indices based on parameters
3. Extracts the relevant data subset
4. Creates index values (1, 2, 3, ...)
5. Calculates means and sums
6. Computes slope and intercept
7. Generates predicted values

Parameters:

- regressionSource: Numeric vector of data points
- regressionLength: Number of data points to use
- regressionOffset: Offset from the end

Return Value

- Type: list
- Components:
 - o slope: numeric (trend direction)
 - o intercept: numeric (baseline value)
 - o predicted_values: numeric vector (fitted values)

Example

r

```
data <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
```

```
result <- linreg(data, regressionLength = 5, regressionOffset = 2)
```

Output: Slope = 3, Intercept = 19, Predicted = 22, 25

Use Cases

- Trend detection
- Price prediction
- Support/resistance levels
- Regression channel construction

Interpretation

Component	Meaning
Slope > 0	Uptrend
Slope < 0	Downtrend
Slope = 0	Sideways trend

Component	Meaning
Larger slope	Stronger trend

7. Relative Strength Index (RSI)

File: rsi.R

Description

RSI measures the magnitude of recent price changes to evaluate overbought or oversold conditions. Values range from 0 to 100, with 70+ indicating overbought and 30- indicating oversold.

Formula

text

$RS = \text{Average Gain} / \text{Average Loss}$

$RSI = 100 - (100 / (1 + RS))$

Implementation Details

The function:

1. Calculates differences between consecutive values
2. Separates gains and losses
3. Calculates initial averages for the first period
4. Applies Wilder's smoothing for subsequent periods
5. Computes RSI values

Parameters:

- data: Numeric vector of data points
- period: Integer for the RSI period

Return Value

- Type: numeric vector
- Length: same as input data

- NA Values: First period values are NA

Example

r

```
data <- c(45, 50, 48, 55, 52, 49, 58, 60, 65, 62)
```

```
result <- rsi(data, period = 5)
```

```
# Output: NA NA NA NA NA 50.52632 68.92562 71.83521 78.21079 66.85966
```

Use Cases

- Overbought/oversold detection
- Divergence analysis
- Momentum measurement
- Trading signals

Interpretation

RSI Range	Interpretation
> 70	Overbought (potential sell)
50 - 70	Bullish territory
30 - 50	Bearish territory
< 30	Oversold (potential buy)

Wilder's Smoothing

The implementation uses Wilder's smoothing method where:

text

Average Gain = (Previous Average Gain × (Period-1) + Current Gain) / Period

Average Loss = (Previous Average Loss × (Period-1) + Current Loss) / Period

8. Stochastic RSI (StochRSI)

File: stochrsi.R

Description

StochRSI applies the stochastic oscillator formula to RSI values, measuring where RSI currently sits within its high-low range over a specified period. It's more sensitive than standard RSI.

Dependencies

- Requires rsi() function from rsi.R
- Requires sma() function from sma.R

Formula

text

$$\text{StochRSI} = (\text{Current RSI} - \text{Min RSI}) / (\text{Max RSI} - \text{Min RSI})$$

$$\%K = \text{SMA}(\text{StochRSI}, k_period)$$

$$\%D = \text{SMA}(\%K, d_period)$$

Implementation Details

The function:

1. Calculates RSI values
2. Removes NA values for min/max calculation
3. Normalizes RSI to 0-1 range
4. Calculates %K as SMA of normalized RSI
5. Calculates %D as SMA of %K

Parameters:

- data: Numeric vector of data points
- period: Integer for RSI period
- k_period: Integer for %K SMA period
- d_period: Integer for %D SMA period

Return Value

- Type: list
- Components:

- o k_line: numeric vector (%K values)
- o d_line: numeric vector (%D values)

Example

r

```
data <- c(45, 50, 48, 55, 52, 49, 58, 60, 65, 62)
```

```
result <- stoch_rsi(data, period = 5, k_period = 3, d_period = 3)
```

Output: List with k_line and d_line

Use Cases

- Enhanced overbought/oversold detection
- Quick price reversal identification
- Momentum confirmation

Interpretation

StochRSI Range	Interpretation
> 0.8	Overbought
0.2 - 0.8	Neutral
< 0.2	Oversold
%K crosses above %D	Bullish signal
%K crosses below %D	Bearish signal

9. Crossover Detection

File: crossover.R

Description

Detects when one array crosses above (bullish) or below (bearish) another array. Essential for generating trading signals from indicator crossovers.

Logic

The function checks for crossovers between consecutive data points:

- "Up": Current arr1 > current arr2 AND previous arr1 ≤ previous arr2
- "Down": Current arr1 < current arr2 AND previous arr1 ≥ previous arr2
- "None": No crossover

Implementation Details

The function:

1. Validates arrays have the same length
2. Initializes signal vector with "None" for first element
3. Checks each subsequent point for crossovers
4. Returns signal vector

Parameters:

- arr1: First numeric vector
- arr2: Second numeric vector (must be same length)

Return Value

- Type: character vector
- Values: "Up", "Down", "None"
- Length: same as input arrays

Example

r

```
arr1 <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
```

```
arr2 <- c(18, 20, 22, 18, 15, 12, 10, 11, 13)
```

```
result <- crossover(arr1, arr2)
```

```
# Output: "None" "None" "None" "Up" "None" "None" "None" "None" "None"
```

Use Cases

- MACD signal line crossovers
- Price vs. moving average breakouts
- Golden/death cross detection
- StochRSI %K and %D crossovers

10. Crossunder Detection

File: crossunder.R

Description

Detects when one array crosses below another array. Bearish signal indicator for technical analysis.

Logic

The function checks for crossunders between consecutive data points:

- TRUE: Current $arr1 < current\ arr2$ AND previous $arr1 \geq previous\ arr2$
- FALSE: No crossunder

Implementation Details

The function:

1. Validates arrays have the same length
2. Initializes signal vector with FALSE for first element
3. Checks each subsequent point for crossunders
4. Returns logical vector

Parameters:

- arr1: First numeric vector
- arr2: Second numeric vector (must be same length)

Return Value

- Type: logical vector
- Values: TRUE (crossunder detected), FALSE (no crossunder)

- Length: same as input arrays

Example

r

```
arr1 <- c(25, 23, 20, 18, 15, 12, 10, 14, 18, 20, 22)
```

```
arr2 <- c(15, 18, 20, 20, 18, 15, 12, 12, 15, 18, 20)
```

```
result <- crossunder(arr1, arr2)
```

Output: FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE

Use Cases

- Bearish divergence detection
- Price breakdown confirmation
- Signal line crossunders
- Support level breakdowns

Common Errors and Solutions

Error Message	Solution
"Data length should be greater than or equal to..."	Increase data length or decrease period parameter
"Both arrays should have the same length"	Ensure arr1 and arr2 have equal lengths
"short_period should be less than long_period"	Adjust MACD parameters
"regressionOffset must be less than regressionLength"	Adjust linear regression parameters
NA values in output	This is normal for RSI and StochRSI (first period values)

